

Module 3

Robertson, Rashai

Bellevue University, CSD-380

Assignment 3.2

June 21, 2026

Version control is one of the most important tools used in modern software development. It allows developers to track changes to source code, collaborate with team members, and restore previous versions of a project when necessary. Systems like Git have become essential components of software engineering because they provide a structured way to manage code changes and reduce the risk of lost work. According to GitHub, version control helps teams maintain a complete history of changes while enabling collaboration and preventing conflicting edits (GitHub, 2024).

Three sources that I found that give guidance on version control are GitHub, Microsoft, and AWS. These three sources go in depth with what you should do when using version control while you create code.

GitHub emphasizes practices like setting up repositories with proper security and backup measures, writing clear commit messages, and making frequent small commits. Developers should test code before committing changes, only commit code that is ready for integration, and regularly update their local copies to avoid conflicts. Using branches allows teams to work on features and fixes independently, while limiting repository access helps protect the project from unauthorized changes.

Microsoft recommends using version control as the central repository for all project files. According to Microsoft, developers should commit changes regularly, create branches for new features, and use pull requests to review code before merging it into the main branch. Microsoft also emphasizes documenting changes through meaningful commit messages and maintaining a stable main branch that is always deployable (Microsoft, 2022).

Another important recommendation is integrating code frequently. By merging smaller changes regularly, teams can reduce conflicts and identify problems earlier in the development process.

Docuwares article states that effective version control depends on clear procedures and consistent document management practices. Organizations should establish standard operating procedures using consistent naming conventions and version numbers and implement access controls to protect document integrity. Additional best practices include maintaining audit logs, using automated approval workflows, encouraging comments and annotations for collaboration, and storing documents in a centralized location. Organizations should also develop archiving and retention policies to ensure compliance with legal and regulatory requirements. Following these practices improves transparency, collaboration, security, and the overall management of documents throughout their lifecycle (Allapanda, 2025).

When comparing these three sources, there are several similarities. GitHub, Microsoft, and DocuWare all emphasize the importance of maintaining a secure repository, using clear commit messages, controlling access to the repository, and creating an organized workflow for collaboration. They also stress the importance of keeping accurate records of changes so that developers can track project history and quickly recover from mistakes. While GitHub and Microsoft focus primarily on software development practices such as branching, pull requests, and code integration, DocuWare focuses more on document management, compliance, and organizational procedures. Despite these differences, all three sources share the goal of improving collaboration, reducing errors, and maintaining project integrity.

Most of the guidelines discussed by the three sources are still relevant today. However, some organizations historically relied on manual approval processes and maintaining extensive documentation outside of version control systems. Modern development teams increasingly use automated workflows, continuous integration, and cloud-based repositories to streamline these processes. As a result, some manual practices have become less necessary than they were in the past.

Based on my research and knowledge, the most important version control guidelines are committing code frequently, writing clear commit messages, using branches for new features, performing code reviews before merging, and limiting repository access to authorized users. I selected these guidelines because they directly improve collaboration and help prevent costly mistakes. Frequent commits make it easier to identify problems, while clear commit messages help team members understand changes. Branches allow developers to work independently without disrupting the main codebase, and code reviews help catch errors before they reach production. Finally, limiting repository access improves security and protects the integrity of the project. Together, these guidelines create a more efficient and reliable software development environment.

In conclusion, version control remains a critical component of modern software development. The guidelines provided by GitHub, Microsoft, and DocuWare demonstrate how effective version control practices improve collaboration, security, and project management. By following best practices such as frequent commits, clear documentation, proper branching, and controlled access, development teams can create higher-quality software while reducing risks and improving productivity.

References:

Allapanda, K. (2025, April 16). *What is version control and why is it important?* DocuWare.

<https://start.docuware.com/blog/document-management/what-is-version-control-why-is-it-important>

GitHub. (2024, July 29). *What is version control?*

https://github.com/resources/articles/what-is-version-control?utm_source=chatgpt.com

Microsoft. (2022, November 28). *About version control systems.* Microsoft Learn.

<https://learn.microsoft.com/en-us/devops/develop/git/what-is-version-control>